



CheesePay × Bridge Integration Architecture

Technical architecture document for integrating Bridge's payment orchestration API into the CheesePay platform — enabling stablecoin on/off-ramps across Kenya, Ghana, Rwanda, and Ethiopia.

PREPARED BY

DATE

VERSION

GRANT TRACK

Stellar Community Fund · Integration Track

Current System Architecture

CheesePay is a multi-chain stablecoin wallet serving African markets with fiat on/off-ramps, P2P transfers, bill payments, and merchant services. The current production architecture spans Frontend, a Backend, and integrations across Blockchain.



Current limitation: Users in Kenya, Ghana, Rwanda, and Ethiopia can hold and transfer USDC but cannot convert to local currency. The Bridge integration addresses this gap by adding fiat on/off-ramps for these markets.

- › **Blockchain** — Stellar + 8 EVM networks with per-chain wallet factory contracts
- › **Backend** — Send USDC by @username or blockchain address
- › **Integrations** — Airtime, data, electricity, TV via Flutterwave
- › **Blockchain** — Veil shielded transactions with snarkjs / Circom circuits
- › **Frontend** — API keys, payment requests, webhook deliveries, multi-currency settlement

Bridge Integration Architecture

Bridge plugs into the existing architecture as the for converting between USDC and local African currencies. The NestJS backend gains a new BridgeModule that handles multi-country fiat conversion across all target markets.



Module

BridgeModule

NestJS module encapsulating all Bridge API interactions — client, service, controller, and webhook handler.

Service

BridgeClient

HTTP client wrapping Bridge REST API with API key auth, request signing, retry logic, and error mapping.

Service

BridgeService

Orchestrates on-ramp and off-ramp flows — coordinates between user wallets, the Bridge API, and transaction records.

Entity

BridgeTransfer

TypeORM entity tracking Bridge transfer lifecycle — maps to existing Transaction entity via foreign key.

Design principle: Bridge is integrated as a modular service. The BridgeModule follows the same patterns as existing service modules, making it straightforward to extend with additional countries as Bridge expands coverage.

Off-Ramp Flow

The off-ramp flow converts a user's USDC into local currency deposited to their bank account via Bridge. This enables withdrawals in KES, GHS, RWF, and ETB.

USDC → LOCAL BANK PAYOUT VIA BRIDGE

1

User Initiates Withdrawal

User selects destination country and enters amount in local currency. Frontend calls `POST /banks/transfer` with country code and bank details.

2

Rate Quote & Confirmation

BridgeService fetches a live FX quote from Bridge for the requested currency pair (USDC → local). The quoted rate and fees are displayed to the user for confirmation.

3

USDC Locked & Transfer Created

BridgeService debits the user's USDC balance (internal ledger hold), creates a `BridgeTransfer` record with status `pending`, and a parent `Transaction` record.

4

Bridge API Call — Create Transfer

BridgeClient sends `POST /v0/transfers` to Bridge with source (USDC on Stellar), destination (local bank), amount, and beneficiary details. Bridge returns a `transfer_id`.

5

USDC Settlement to Bridge

Backend sends the USDC from the CheesePay Stellar treasury account to Bridge's designated Stellar address. The Stellar transaction hash is recorded on the BridgeTransfer.

6

Bridge Executes Local Payout

Bridge converts USDC to local currency at the locked rate and initiates the bank transfer through its local banking partner network.

7

Webhook Confirmation

Bridge sends a webhook (`transfer.completed`) to `POST /webhooks/bridge`. The handler updates the BridgeTransfer and parent Transaction to `completed`. User receives a push notification.

- › If Bridge API returns an error at step 4, USDC hold is released and the Transaction is marked `failed`
- › If the Stellar payment fails at step 5, the BridgeTransfer is cancelled via `DELETE /v0/transfers/:id`
- › If Bridge webhook reports `transfer.failed`, USDC is refunded to the user's balance and a failure notification is sent

- › Idempotency keys prevent duplicate transfers on network retries

On-Ramp Flow

The on-ramp flow allows users to deposit local currency and receive USDC. Bridge handles the local currency collection and USDC settlement.

LOCAL CURRENCY → USDC VIA BRIDGE

1

User Requests Deposit

User selects "Add Money" and chooses their local currency (KES, GHS, RWF, or ETB). Frontend calls `POST /wallet/deposit/bridge` with the amount in local currency.

2

Bridge Payment Link Created

BridgeClient calls `POST /v0/transfers` with source as local currency and destination as USDC on Stellar. Bridge returns payment instructions (bank details or mobile money info) for the user to complete.

3

User Completes Local Payment

User transfers local currency to the provided bank or mobile money details. This happens outside the CheesePay app via the user's local banking app or mobile money wallet.

4

Bridge Detects Payment

Bridge confirms receipt of local currency funds through its banking partners. Webhook `transfer.payment_received` is sent to CheesePay. BridgeTransfer status updates to `processing`.

5

USDC Settled to CheesePay

Bridge converts the local currency to USDC and sends it to the CheesePay Stellar treasury address. The Stellar transaction is detected by the backend's deposit monitor.

6

USDC Credited to User

Backend credits the USDC to the user's internal balance, creates a `deposit` Transaction record, and sends a push notification confirming the deposit.

COUNTRY	CURRENCY	COLLECTION METHODS
	KES	M-Pesa, bank transfer
	GHS	Mobile money, bank transfer
	RWF	Mobile money, bank transfer
	ETB	Bank transfer

Country Coverage & Routing

Bridge serves as the unified payment rail for all four target markets. The `BridgeService` uses the user's country code to determine the correct currency and available payment methods for each transaction.

```
// bridge.service.ts - country configuration

const SUPPORTED_COUNTRIES = {
  'KE': { currency: 'KES', methods: ['mpesa', 'bank_transfer'] },
  'GH': { currency: 'GHS', methods: ['mobile_money', 'bank_transfer'] },
  'RW': { currency: 'RWF', methods: ['mobile_money', 'bank_transfer'] },
  'ET': { currency: 'ETB', methods: ['bank_transfer'] },
};

function createTransfer(countryCode: string, amount: number) {
  const config = SUPPORTED_COUNTRIES[countryCode];
  if (!config) throw new UnsupportedCountryError(countryCode);

  return this.bridgeClient.createTransfer({
    source_currency: 'usdc',
    destination_currency: config.currency,
    destination_payment_rail: config.methods[0],
    amount,
  });
}
```

Kenya

Provider: Bridge

Currency: KES

On-Ramp: M-Pesa / bank transfer

Off-Ramp: Bank payout

Bridge

Ghana

Provider: Bridge

Currency: GHS

On-Ramp: Mobile money / bank

Off-Ramp: Bank payout

Bridge

Rwanda

Provider: Bridge

Currency: RWF

On-Ramp: Mobile money / bank

Off-Ramp: Bank payout

Bridge

Ethiopia

Provider: Bridge

Currency: ETB

On-Ramp: Bank transfer

Off-Ramp: Bank payout

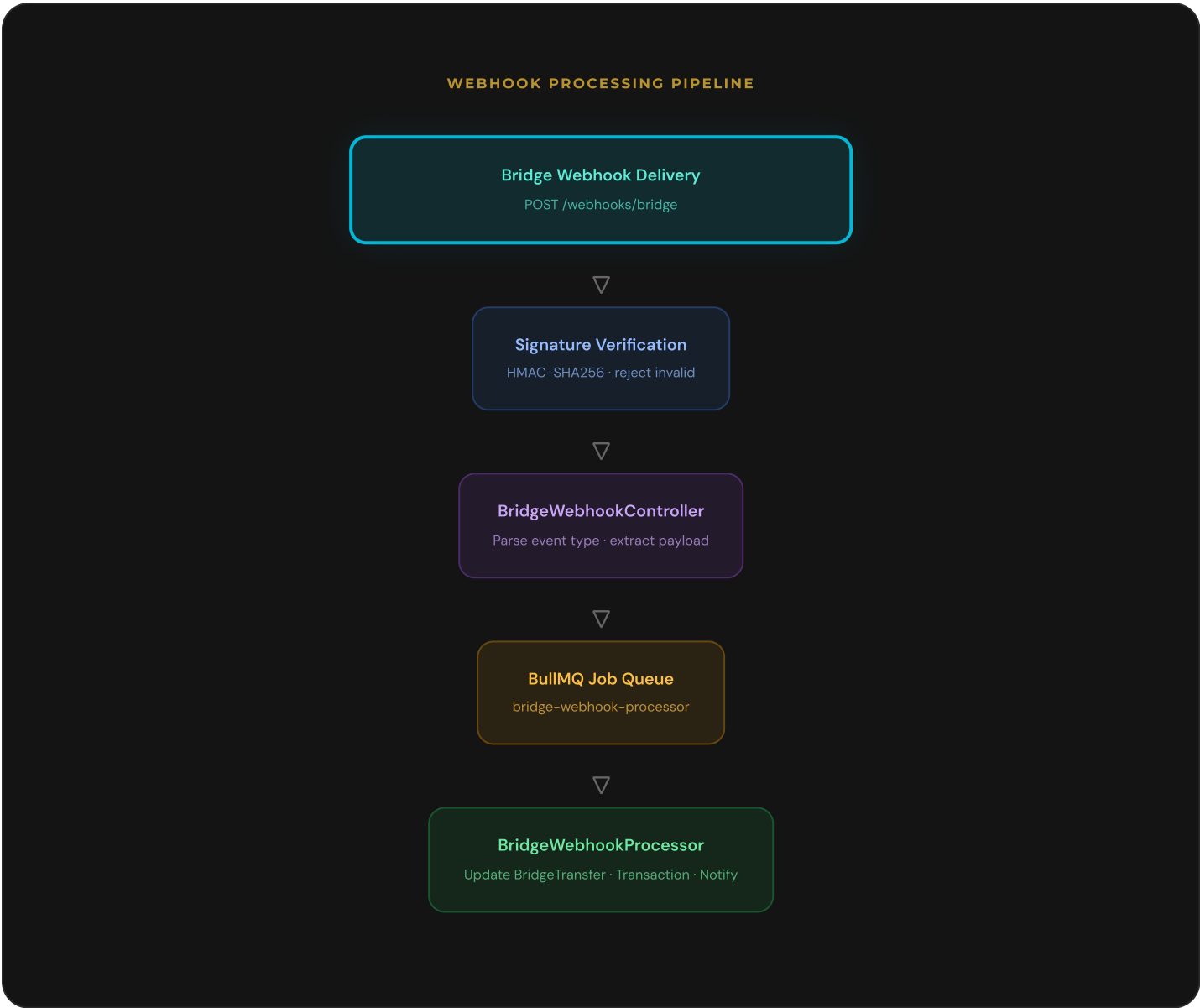
Bridge

Extensibility: Adding a new country requires only a new entry in the `SUPPORTED_COUNTRIES` map. As Bridge expands coverage to additional African markets, CheesePay can onboard new countries with minimal code changes.

- › Bridge provides and locks the FX rate at transfer creation time — CheesePay displays the Bridge-quoted rate to the user before confirmation
- › Rates are cached in Redis with a 60-second TTL to minimize API calls while keeping quotes fresh
- › The locked rate is stored on the `BridgeTransfer` entity for audit and reconciliation
- › Users see the exact amount they will receive in local currency before confirming the transaction

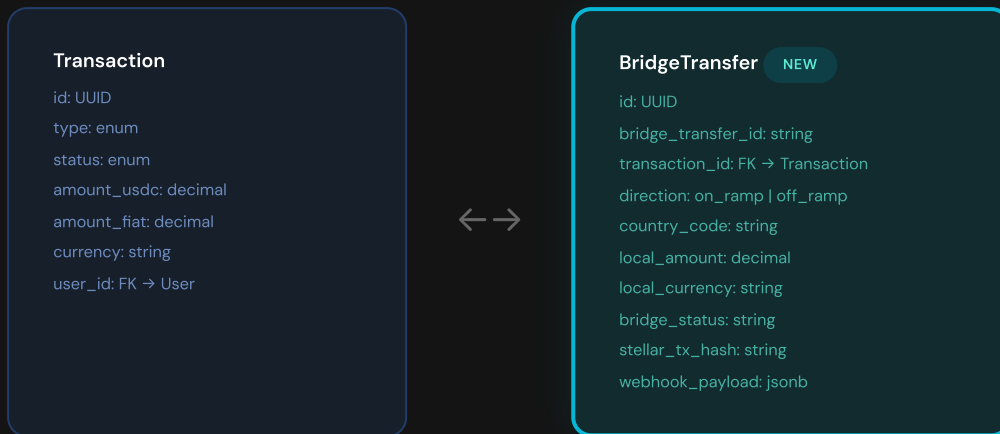
Data Flow & Webhook Architecture

Bridge webhooks integrate into CheesePay's existing event-driven architecture. The system uses internal events and for for async processing.



BRIDGE EVENT	ACTION	USER NOTIFICATION
transfer.created	Log transfer initiation, update status to processing	—
transfer.payment_received	Update BridgeTransfer to payment_received	Push: "Payment received, converting..."
transfer.completed	Mark Transaction completed , credit/confirm funds	Push: "Transfer complete!"
transfer.failed	Mark Transaction failed , release held USDC	Push: "Transfer failed — funds returned"
transfer.refunded	Credit refunded USDC back to user balance	Push: "Refund received"

ENTITY RELATIONSHIP — BRIDGE EXTENSION



- › Each webhook is stored with its `event_id` — duplicate deliveries are detected and skipped
- › Each transfer request includes a client-generated idempotency key tied to the Transaction UUID
- › BullMQ jobs retry up to 3 times with exponential backoff (1s, 4s, 16s) on transient failures
- › A scheduled job polls `GET /v0/transfers/:id` for any transfers stuck in `processing` for >30 minutes

Tech Stack Summary

Complete technology inventory for the CheesePay platform including the Bridge integration components.

LAYER	TECHNOLOGY	PURPOSE
Backend	NestJS 11	API server, modular architecture, dependency injection
	PostgreSQL + TypeORM	Primary data store, entity management, migrations
	Redis + BullMQ	Rate caching, job queues, session store
	Next.js 14 + React 18	Progressive web app with TailwindCSS, Zustand, React Query
	React Native + Expo 56	iOS and Android with biometric auth, secure storage
	Stellar SDK + Horizon	USDC wallets, payments, anchor integrations
	Soroban (Stellar)	Fee pool contract, on-chain logic
	ethers.js v6	Multi-chain wallets across 8 EVM networks
	Circom + snarkjs	Zero-knowledge proofs for shielded transactions (Veil)
	Bridge API NEW	USDC on/off-ramps for KES, GHS, RWF, ETB markets
Frontend	Flutterwave API	Airtime, data, electricity, TV across markets
	Dojah API	Identity verification, selfie matching
	Resend	Transactional emails with custom HTML templates
	Expo Server SDK + web-push	Mobile and web push notifications
	Telegram Bot API	Admin alerts for treasury and security events
	JWT + Passport + bcrypt	Access/refresh tokens, device signatures, PIN hashing
	Swagger / OpenAPI	Auto-generated API documentation
Integration	Socket.IO	Live balance updates, transaction notifications

Stellar

USDC · Soroban

Base

USDC · USDT

Arbitrum

USDC · USDT

Polygon

USDC · USDT

Celo

USDC · USDT

Ethereum

USDC · USDT

Optimism

USDC · USDT

BNB Chain

USDC · USDT

Lisk

USDC · USDT

- › HMAC-SHA256 device binding prevents credential replay from unknown devices
- › Client-side HMAC hashing with userID salt, server-side bcrypt — PIN never transmitted in plaintext
- › Stellar secret keys encrypted at rest with AES-256 using a per-deployment encryption key
- › All inbound Bridge webhooks validated via HMAC signature before processing
- › NestJS Throttler guards on auth and transaction endpoints
- › KYC tier determines transaction and withdrawal limits (Silver → Gold → Black)



CheesePay

Stablecoin infrastructure for Africa · cheesepay.xyz